WEB-PERFORMANCE-OPTIMIZATION

웹 성능 저하 원인과 프런트엔드 최적화

연관 기출 · 128-3-2

한눈에 이해

페이지가 느린 원인을 서버·네트워크·브라우저로 나누어 측정한 뒤, 사용자가 기다리는 경로부터 줄입니다.

정의와 핵심 원리

웹 성능 최적화는 자원 전송과 브라우저 파싱·실행·렌더링의 병목을 줄여 로딩·상호작용·시각적 안정성을 개선하는 활동입니다.

• 성능 저하는 서버 처리·DB, 왕복지연·대역폭·연결, 큰 이미지·스크립트·폰트, 렌더링 차단 CSS/JS, 과도한 DOM·레이아웃·메인스레드 작업, 캐시 부재와 외부 스크립트에서 발생합니다. 한 계층의 개선만으로 다른 병목까지 제거되지는 않습니다.

• (1)이미지를 적절한 크기·형식·압축과 반응형 소스로 제공합니다. (2)번들 축소·트리셰이킹·코드 분할로 초기 JS를 줄입니다. (3)defer/module 등으로 불필요한 파서 차단을 피하고 중요한 CSS를 우선합니다.

• (4)HTTP 캐시·압축·CDN으로 전송과 반복 로딩 비용을 줄입니다. (5)화면 아래 이미지·iframe은 지연 로딩하되 핵심 이미지까지 늦추지 않습니다. (6)DOM 작업을 모으고 긴 JS 작업을 분리하거나 Worker를 사용하여 입력 반응을 확보합니다.

• 최적화 전후 동일한 기기·회선·부하에서 실제 사용자와 실험실 측정을 함께 비교합니다. LCP, INP, CLS 및 요청 워터폴·롱태스크를 관찰하되 하나의 점수를 서비스 품질 전체로 일반화하지 않습니다.

구조·비교·판단

구분	원리·특징	적용·판단
전송 병목	큰 자원·왕복·캐시 부재	크기 축소·캐시·CDN
실행/렌더링 병목	긴 JS·레이아웃 반복	코드 분할·DOM 최적화·Worker
잘못된 선행로딩	모든 자원을 미리 받음	핵심 경로 우선, 사용하지 않는 자원은 지연

적용 예제

강사 창작 적용 예시: 모바일 상품 목록

강사 생성 학습 예제입니다. 공식 기준답안이 아닙니다.

조건: 원본 사진 30장과 큰 검색 스크립트를 초기 로딩합니다.

1. 워터폴로 이미지 전송량과 JS 긴 작업을 확인합니다.

2. 보이는 사진은 알맞은 크기로 우선 제공하고 아래 사진은 지연합니다.

3. 상세검색 코드를 분할하고 저성능 기기에서 상호작용을 재측정합니다.

결과·해석: 어떤 비용을 줄였는지 원인과 지표로 설명할 수 있습니다.

전제·주의: 성능 개선 수치는 측정 전에는 제시하지 않습니다. 무분별한 캐시는 개인정보와 최신성 문제를 만들 수 있습니다.

답안 골격과 혼동 방지

저하 원인 계층별 분류 → 프런트엔드 6개 방안과 작동원리 → 측정→개선→재측정 사례

- 모든 이미지를 lazy-load하면 주요 콘텐츠가 더 늦어질 수 있습니다.
- 압축만으로 JS 실행시간이 줄어드는 것은 아닙니다.

내용을 가리고 회상

1. 전송시간과 실행시간을 어떻게 구분합니까?

2. 코드 분할은 어떤 초기 비용을 줄입니까?

3. 실험실 점수와 실제 사용자 지표가 다른 이유는 무엇입니까?

적용 기준과 근거

고전 개념의 설명이며 2026-11-05 열람 원문을 기준으로 했습니다.

내용 검토 상태: 출처 확인. 출처 확인은 공식 정답이나 학습 성취를 뜻하지 않습니다.

출처별 상세 열람 범위는 위키의 출처 기록에서 확인합니다.

- Learn Performance - Google web.dev; 지속 갱신 문서; 확인 2026-11-05.